



FORENSIC LAB REPORT

DIGITAL FORENSICS / LAB 01

Data Carving and File Recovery

Investigating raw file content and recovering deleted or embedded files from forensic disk images.

LAB TITLE

Digital Forensics Lab: Data Carving and File Recovery

STUDENT NAME

Almustapha Yusuf

STUDENT ID

fwsd2511343@students.icdfa.edu.ng

COURSE

Digital Forensics

INSTRUCTOR

Aminu Idris

SUBMISSION

05/09/2026 · Version 1.0

2.2 Executive Summary

This lab focused on examining raw file content and recovering deleted or embedded files from forensic images when file-system metadata is missing, damaged, or unreliable. The investigation utilized hex analysis tools (xxd), The Sleuth Kit (TSK), Binwalk, and Scalpel to systematically analyze a JPEG sample, a FAT12 superfloppy image, and a corrupted FAT16 USB image. Key activities included identifying JPEG Start of Image (SOI) and End of Image (EOI) signatures, reconstructing a JPEG from a hex dump, examining metadata strings, extracting embedded content from a DOCX file, and carving deleted files from a USB image.

The investigation revealed that the USB image's filesystem was encrypted or corrupted (high entropy), which rendered standard filesystem browsing ineffective. However, raw data carving successfully recovered 17 JPEG files, demonstrating the critical role of carving when filesystem metadata is unreliable. The lab reinforced the principle that file signatures, not filesystem tables, are the ground truth for file recovery in adversarial or degraded forensic environments.

KEY OUTCOME

17 JPEG files (8 Exif, 9 JFIF) carved from a high-entropy (7.58) FAT16 USB image using Scalpel, despite the filesystem being unreadable by standard TSK tools.

2.3 Lab Objectives

The primary objectives of this lab were designed to build a systematic understanding of how digital evidence can be recovered when traditional filesystem metadata is unavailable. Each objective maps to a concrete skill required in real-world digital forensics investigations, from foundational binary analysis through to advanced carving techniques on damaged or encrypted media:

- Explain why data carving is required when file-system metadata is missing or unreliable.
- Use **xxd** to examine binary file content and identify headers and footers.
- Recognize JPEG **SOI** (Start of Image) and **EOI** (End of Image) signatures and reconstruct binary content from a hexadecimal dump.
- Use **The Sleuth Kit** to examine inode, block, and sector-level data.
- Identify and extract embedded content using **Binwalk**.
- Recover deleted files from a forensic USB image using **Scalpel**.
- Verify recovered evidence using cryptographic hashes (MD5, SHA-256).
- Prepare a professional forensic report containing commands, screenshots, findings, and limitations.

2.4 Tools and Resources Used

All tools were operated within a Kali Linux environment, the de-facto platform for forensic and penetration testing workloads. Each tool listed below was selected for a specific analytical purpose: xxd for low-level binary

inspection, The Sleuth Kit for filesystem-aware analysis, Binwalk for signature-based embedded content discovery, and Scalpel for raw carving when filesystem metadata is absent or hostile. Cryptographic hashes were computed at every stage to preserve the chain of custody and prove evidentiary integrity.

Tool	Role in Investigation
Kali Linux	Operating system platform hosting all forensic tools used in this lab.
xxd	Hexadecimal analysis and binary reconstruction of JPEG headers and footers.
The Sleuth Kit (TSK)	Filesystem inspection: img_stat, mmls, fsstat, fls, icat, blkcat.
Binwalk	Embedded file signature discovery and extraction (e.g. JPEG inside DOCX).
Scalpel	Raw data carving of deleted files from a corrupted FAT16 USB image.
7-Zip (p7zip)	Extraction of the compressed USB forensic image archive.
MD5 / SHA-256 Hashing	Integrity verification of original evidence and recovered artifacts.

2.5 Methodology

The investigation was conducted in six sequential parts (A through F), each building on the findings of the previous stage. The workflow was designed to mirror the escalation path a forensic analyst follows in practice: begin with low-level binary inspection, progress to filesystem-aware analysis, then fall back to raw carving when filesystem metadata proves unreliable.

Part A — File Signatures and Hex Analysis

A JPEG evidence file was inspected at the byte level to confirm its file signatures. The header and footer of the file were examined using xxd, then a plain hex dump was generated and used to reconstruct the image. Hash comparison verified that the reconstruction was bit-for-bit identical to the original.

- › xxd evidence/J_ub_law.jpg | head -5 → confirmed SOI signature ffd8
- › xxd evidence/J_ub_law.jpg | tail -5 → confirmed EOI signature ffd9
- › xxd -p evidence/J_ub_law.jpg > results/J_ub_law_plain.hex
- › xxd -r -p results/J_ub_law_plain.hex > results/reconstructed.jpg
- › md5sum and sha256sum comparison — both hashes matched perfectly

Part B — Metadata / Strings Examination

The strings utility was used to extract human-readable metadata embedded within the JPEG file. A targeted grep pattern was applied to surface camera make, model, software, and date information that is typically embedded in EXIF segments.

```
strings evidence/J_ub_law.jpg | grep -i -E "camera|software|date|..."
```

Part C — TSK Examination

The Sleuth Kit was used to examine the FAT12 superfloppy image at the inode, block, and sector level. mmls returned no partition table (confirming a raw superfloppy layout), fsstat confirmed FAT12 filesystem parameters, and fls revealed several deleted files recoverable by inode number.

- › mmls evidence/Ch01InChap01.dd → empty output (raw superfloppy, no partition table)
- › fsstat -o 0 evidence/Ch01InChap01.dd → FAT12, sector size 512, root dir at 2
- › fls -o 0 evidence/Ch01InChap01.dd → listed deleted files (e.g. Client Info.mdb inode 5, Billing Letter.doc inode 8)
- › icat -o 0 evidence/Ch01InChap01.dd 5 → extracted Microsoft Access Database
- › blkcat -o 0 evidence/Ch01InChap01.dd 0 → extracted DOS/MBR boot sector

Part D — Binwalk Extraction

Binwalk was used to discover embedded file signatures inside the evidence files. The DOCX file was identified as a ZIP archive containing an embedded JPEG at offset 3882 (ZIP header) — manual extraction via dd required locating the actual JPEG byte signature using grep -abo.

- › binwalk evidence/J_ub_law.jpg / binwalk evidence/Ch01InChap01.dd
- › binwalk evidence/File_carving.docx → ZIP archive with word/media/image1.jpeg at offset 3882
- › grep -abo '\$\xff\xd8\xff' evidence/File_carving.docx → actual JPEG offset 3934
- › dd if=evidence/File_carving.docx of=results/manual_extracted_image1.jpeg bs=1 skip=3934 count=85046
- › binwalk -e evidence/File_carving.docx -C results/binwalk_extracted → automatic extraction

Part E — USB Image Preparation

A 120 MB compressed USB forensic image was extracted using 7-Zip. Hashes were computed and recorded before analysis. mmls identified a Win95 FAT16 partition starting at sector 128 (byte offset 65536). However, fsstat reported high entropy (7.58), indicating the filesystem was encrypted or corrupted and could not be browsed through standard metadata tools.

- › 7z x evidence/120M.7z -oevidence/ → evidence/120M/usb_fat_carving.001
- › md5sum / sha256sum calculated and recorded
- › mmls evidence/120M/usb_fat_carving.001 → Win95 FAT16 partition at sector 128
- › fsstat -o 65536 evidence/120M/usb_fat_carving.001 → "Possible encryption detected (High entropy 7.58)"

Part F — Scalpel Carving

Scalpel was installed and configured with JPEG rules enabled in scalpel.conf. A dedicated output directory was created, and Scalpel was run against the raw USB image. 17 JPEG files (8 Exif, 9 JFIF) were carved and verified using the file command, proving that raw carving bypasses filesystem encryption entirely.

- › grep -n "jpg" scalpel.conf → uncommented JPEG rules
- › mkdir -p results/scalpel_output
- › scalpel -c scalpel.conf -o results/scalpel_output evidence/120M/usb_fat_carving.001
- › Carved 17 files (8 Exif, 9 JFIF) — verified with file command

2.6 Screenshots and Evidence

The following screenshots document the critical stages of the investigation. Each figure is captioned with the forensic tool shown and the corresponding methodology part. Together they form the visual evidence trail that supports the findings in Section 2.7.

```
(kali@kali)-[~/ICDFA-Forensics-Lab]
$ cp ~/Downloads/Ch01InChap01.dd ~/Downloads/120M.7z ~/ICDFA-Forensics-Lab/evidence/

(kali@kali)-[~/ICDFA-Forensics-Lab]
$ ls -lh evidence/
total 39M
-rwxrwxr-- 1 kali kali 36M Sep  4 16:58 120M.7z
-rwxrwxr-- 1 kali kali 1.5M Sep  4 16:58 Ch01InChap01.dd
-rwxrwxr-- 1 kali kali 1.7M Sep  4 16:58 J_ub_law.jpg

(kali@kali)-[~/ICDFA-Forensics-Lab]
$ md5sum evidence/Ch01InChap01.dd evidence/J_ub_law.jpg evidence/120M.7z > results/hash_pre.txt

(kali@kali)-[~/ICDFA-Forensics-Lab]
$ sha256sum evidence/Ch01InChap01.dd evidence/J_ub_law.jpg evidence/120M.7z >> results/hash_pre.txt

(kali@kali)-[~/ICDFA-Forensics-Lab]
$ cat results/hash_pre.txt
a117773bcf1fc88ec0ab8e0a349fbbcb evidence/Ch01InChap01.dd
83a360ac7f7e0ca318e5bfe39f95f137 evidence/J_ub_law.jpg
dfe7b5424e54cd1bf50d5df47aceeb3c evidence/120M.7z
3ce8053e4f3d9c8ab98b3aadb2480685efb8e4980d34297b83bd5a09b1a7b122 evidence/Ch01InChap01.dd
238ff34393c50e52c0e8b14fcff8ec7dc29e23914dbc435f8ef998d172a91468 evidence/J_ub_law.jpg
2d2a3d93c9ec65bcad9f89c9894429ea72caa8add07726a3b96ec9a6ab6a58ce evidence/120M.7z

(kali@kali)-[~/ICDFA-Forensics-Lab]
$ xxd evidence/J_ub_law.jpg | head -5
00000000: ffd8 ffe1 17f0 4578 6966 0000 4949 2a00 .....Exif..II*.
00000010: 0800 0000 1000 0001 0300 0100 0000 4013 .....@.
00000020: 0000 0101 0300 0100 0000 d00c 0000 0201 .....
00000030: 0300 0300 0000 ce00 0000 0601 0300 0100 .....
00000040: 0000 0200 0000 0f01 0200 1200 0000 d400 .....

(kali@kali)-[~/ICDFA-Forensics-Lab]
$ xxd evidence/J_ub_law.jpg | head -5 | tee results/header_hex.txt
00000000: ffd8 ffe1 17f0 4578 6966 0000 4949 2a00 .....Exif..II*.
00000010: 0800 0000 1000 0001 0300 0100 0000 4013 .....@.
00000020: 0000 0101 0300 0100 0000 d00c 0000 0201 .....
00000030: 0300 0300 0000 ce00 0000 0601 0300 0100 .....
```

Figure 1 — Folder structure and hash baseline: `ls -lh evidence/` shows the three evidence files (120M.7z 36M, Ch01InChap01.dd 1.5M, J_ub_law.jpg 1.7M), then `md5sum` and `sha256sum` are computed for all three and saved to `results/hash_pre.txt` (Part A — integrity baseline).

```

(kali@kali)-[~/ICDFA-Forensics-Lab]
$ xxd evidence/J_ub_law.jpg | head -5
00000000: ffd8 ffe1 17f0 4578 6966 0000 4949 2a00 .....Exif..II*.
00000010: 0800 0000 1000 0001 0300 0100 0000 4013 .....@.
00000020: 0000 0101 0300 0100 0000 d00c 0000 0201 .....
00000030: 0300 0300 0000 ce00 0000 0601 0300 0100 .....
00000040: 0000 0200 0000 0f01 0200 1200 0000 d400 .....

(kali@kali)-[~/ICDFA-Forensics-Lab]
$ xxd evidence/J_ub_law.jpg | head -5 | tee results/header_hex.txt
00000000: ffd8 ffe1 17f0 4578 6966 0000 4949 2a00 .....Exif..II*.
00000010: 0800 0000 1000 0001 0300 0100 0000 4013 .....@.
00000020: 0000 0101 0300 0100 0000 d00c 0000 0201 .....
00000030: 0300 0300 0000 ce00 0000 0601 0300 0100 .....
00000040: 0000 0200 0000 0f01 0200 1200 0000 d400 .....

(kali@kali)-[~/ICDFA-Forensics-Lab]
$ xxd evidence/J_ub_law.jpg | tail -5
0019d1b0: 294f ae4a a859 c9b4 2885 515a f676 3a80 )0.J.Y..(.QZ.v:.
0019d1c0: fc7f 81df 1c79 f5e1 dc0f 53e3 a4d5 5f45 .....y....S..._E
0019d1d0: 182a d52d 2406 abc1 ad69 9479 0789 d1e1 .*.-$....i.y....
0019d1e0: ff00 5254 0938 040e 6ff5 f75a 9a54 f565 ..RT.8...o..Z.T.e
0019d1f0: d351 9ebf ffd9 .Q....

(kali@kali)-[~/ICDFA-Forensics-Lab]
$ xxd evidence/J_ub_law.jpg | tail -5 | tee results/footer_hex.txt
0019d1b0: 294f ae4a a859 c9b4 2885 515a f676 3a80 )0.J.Y..(.QZ.v:.
0019d1c0: fc7f 81df 1c79 f5e1 dc0f 53e3 a4d5 5f45 .....y....S..._E
0019d1d0: 182a d52d 2406 abc1 ad69 9479 0789 d1e1 .*.-$....i.y....
0019d1e0: ff00 5254 0938 040e 6ff5 f75a 9a54 f565 ..RT.8...o..Z.T.e
0019d1f0: d351 9ebf ffd9 .Q....

(kali@kali)-[~/ICDFA-Forensics-Lab]
$ xxd -p evidence/J_ub_law.jpg > results/J_ub_law_plain.hex

(kali@kali)-[~/ICDFA-Forensics-Lab]
$ xxd -r -p results/J_ub_law_plain.hex > results/reconstructed.jpg

```

Figure 2 — xxd evidence/J_ub_law.jpg | head -5 confirming JPEG Start of Image (SOI) signature ffd8 followed by ffe1 (Exif marker). Output was also tee'd to results/header_hex.txt for evidentiary preservation (Part A).

Figure 3 — xxd evidence/J_ub_law.jpg | tail -5 confirming JPEG End of Image (EOI) signature ffd9. Together with Figure 2, this establishes the byte boundaries of the JPEG file format (Part A).

Figure 4 — MD5 and SHA-256 hash comparison between the original evidence/J_ub_law.jpg and the reconstructed results/reconstructed.jpg. Both MD5 (83a360ac7f7e0ca318e5bfe39f95f137) and SHA-256 (238ff34393c50e52...) hashes match perfectly, proving the xxd round-trip reconstruction is bit-for-bit identical to the original (Part A — integrity verification).

```

└─$ xxd evidence/J_ub_law.jpg | head -20 > results/metadata_hex.txt
cat results/metadata_hex.txt
00000000: ffd8 ffe1 17f0 4578 6966 0000 4949 2a00 .....Exif..II*.
00000010: 0800 0000 1000 0001 0300 0100 0000 4013 .....@.
00000020: 0000 0101 0300 0100 0000 d00c 0000 0201 .....
00000030: 0300 0300 0000 ce00 0000 0601 0300 0100 .....
00000040: 0000 0200 0000 0f01 0200 1200 0000 d400 .....
00000050: 0000 1001 0200 0900 0000 e600 0000 1201 .....
00000060: 0300 0100 0000 0100 0000 1501 0300 0100 .....
00000070: 0000 0300 0000 1a01 0500 0100 0000 ef00 .....
00000080: 0000 1b01 0500 0100 0000 f700 0000 2801 .....(
00000090: 0300 0100 0000 0200 0000 3101 0200 2100 .....1...!.
000000a0: 0000 ff00 0000 3201 0200 1400 0000 2001 .....2.....
000000b0: 0000 9882 0200 0c00 0000 3401 0000 9e9c .....4.....
000000c0: 0100 5200 0000 4001 0000 6987 0400 0100 ..R...@...i....
000000d0: 0000 9401 0000 4404 0000 0800 0800 0800 .....D.....
000000e0: 4e49 4b4f 4e20 434f 5250 4f52 4154 494f NIKON CORPORATIO
000000f0: 4e00 4e49 4b4f 4e20 4434 00c0 c62d 0010 N.NIKON D4...-..
00000100: 2700 00c0 c62d 0010 2700 0041 646f 6265 '.....'..Adobe
00000110: 2050 686f 746f 7368 6f70 2032 312e 3120 Photoshop 21.1
00000120: 284d 6163 696e 746f 7368 2900 3230 3230 (Macintosh).2020
00000130: 3a30 383a 3230 2031 303a 3230 3a30 3500 :08:20 10:20:05.

(kali@kali)-[~/ICDFA-Forensics-Lab]
└─$ strings evidence/J_ub_law.jpg > results/all_strings.txt
wc -l results/all_strings.txt
cat results/all_strings.txt | head -50
21307 results/all_strings.txt
Exif
NIKON CORPORATION
NIKON D4
Adobe Photoshop 21.1 (Macintosh)
2020:08:20 10:20:05
HOWARD KORN
0221
2013:10:08 18:56:21
2013:10:08 18:56:21
2027750
17.0-35.0 mm f/2.8
Adobe_CM
Adobe
b34r
7GWgw

```

Figure 5 — strings metadata extraction: xxd | head -20 reveals embedded EXIF metadata, then strings evidence/J_ub_law.jpg | head -50 surfaces Camera Make (NIKON CORPORATION), Camera Model (NIKON D4), Software (Adobe Photoshop 21.1 Macintosh), and capture timestamp (2020:02:08 10:20:05). Total strings extracted: 21307 (Part B).


```

kali@kali: ~/ICDFA-Forensics-Lab
Session Actions Edit View Help
(kali@kali)~-[~/ICDFA-Forensics-Lab]
$ mmls evidence/Ch01InChap01.dd > results/tsk_mmls.txt
cat results/tsk_mmls.txt

(kali@kali)~-[~/ICDFA-Forensics-Lab]
$ fsstat -o 0 evidence/Ch01InChap01.dd > results/tsk_fsstat.txt
cat results/tsk_fsstat.txt

FILE SYSTEM INFORMATION
-----
File System Type: FAT12

OEM Name: ',k0nIHC
Volume ID: 0x0
Volume Label (Boot Sector):
Volume Label (Root Directory):
File System Type Label: *3+*m|

Sectors before file system: 0

File System Layout (in sectors)
Total Range: 0 - 2879
* Reserved: 0 - 0
** Boot Sector: 0
* FAT 0: 1 - 9
* FAT 1: 10 - 18
* Data Area: 19 - 2879
** Root Directory: 19 - 32
** Cluster Area: 33 - 2879

METADATA INFORMATION
-----
Range: 2 - 45782
Root Directory: 2

CONTENT INFORMATION
-----
Sector Size: 512
Cluster Size: 512
Total Cluster Range: 2 - 2848

FAT CONTENTS (in sectors)
-----

```

Figures 6 & 7 — mmls evidence/Ch01InChap01.dd returns empty (confirming raw superfloppy with no partition table), then fsstat -o 0 confirms FAT12 filesystem with sector size 512 and root directory at sector 2 (Part C).

```

kali@kali: ~/ICDFA-Forensics-Lab
Session Actions Edit View Help
Range: 2 - 45782
Root Directory: 2

CONTENT INFORMATION
-----
Sector Size: 512
Cluster Size: 512
Total Cluster Range: 2 - 2848

FAT CONTENTS (in sectors)
-----
33-236 (204) → EOF
285-311 (27) → EOF

(kali@kali)~-[~/ICDFA-Forensics-Lab]
$ fls -o 0 evidence/Ch01InChap01.dd > results/tsk_fls.txt
cat results/tsk_fls.txt
r/r * 5: Client Info.mdb
r/r * 8: Billing Letter.doc
r/r * 11: confirmation.txt
r/r 13: Income.xls
r/r * 15: letter1.txt
r/r * 17: Regrets.doc
v/v 45779: $MBR
v/v 45780: $FAT1
v/v 45781: $FAT2
V/V 45782: $OrphanFiles

(kali@kali)~-[~/ICDFA-Forensics-Lab]
$ icat -o 0 evidence/Ch01InChap01.dd 5 > results/extracted_from_inode.bin
file results/extracted_from_inode.bin
results/extracted_from_inode.bin: Microsoft Access Database

(kali@kali)~-[~/ICDFA-Forensics-Lab]
$ blkcat -o 0 evidence/Ch01InChap01.dd 0 > results/block_data.bin
file results/block_data.bin
results/block_data.bin: DOS/MBR boot sector, code offset 0x34+2, OEM-ID ',k0nIHC' cached by Windows 9M, root entries 224, sectors 2880 (volumes ≤ 32 MB),
sectors/FAT 9, sectors/track 18, dos < 4.0 BootSector (0), FAT (12 bit by descriptor+sectors)

(kali@kali)~-[~/ICDFA-Forensics-Lab]
$

```

Figure 8 — fsstat FAT contents showing cluster/sector ranges, followed by fls -o 0 listing deleted files with their inode numbers: Client Info.mdb (inode 5), Billing Letter.doc (inode 8), and others. The asterisk (*) marks deleted entries (Part C).

```
(kali@kali)-[~/ICDFA-Forensics-Lab]
$ icat -o 0 evidence/Ch01InChap01.dd 5 > results/extracted_from_inode.bin
file results/extracted_from_inode.bin

results/extracted_from_inode.bin: Microsoft Access Database

(kali@kali)-[~/ICDFA-Forensics-Lab]
$ blkcat -o 0 evidence/Ch01InChap01.dd 0 > results/block_data.bin
file results/block_data.bin

results/block_data.bin: DOS/MBR boot sector, code offset 0x34+2, OEM-ID ", 'k0nIHC" cached by Windows 9M, root entries 224, sectors 2880 (volumes ≤32 MB),
sectors/FAT 9, sectors/track 18, dos < 4.0 BootSector (0), FAT (12 bit by descriptor+sectors)
```

Figure 9 — `icat -o 0 evidence/Ch01InChap01.dd 5` extracts the file at inode 5; the file command identifies it as a Microsoft Access Database. `blkcat -o 0 ... 0` extracts block 0, identified by file as a DOS/MBR boot sector with OEM-ID "k@nIHC", 224 root entries, 2880 sectors, and 9 sectors/FAT — confirming FAT12 (Part C).

```
(kali@kali)-[~/ICDFA-Forensics-Lab]
$ which binwalk
/usr/bin/binwalk

(kali@kali)-[~/ICDFA-Forensics-Lab]
$ binwalk evidence/J_ub_law.jpg | tee results/binwalk_jpg.txt
```

DECIMAL	HEXADECIMAL	DESCRIPTION
0	0x0	JPEG image data, EXIF standard
12	0xC	TIFF image data, little-endian offset of first image directory: 8
26844	0x68DC	Copyright string: "Copyright 1999 Adobe Systems Incorporated"

```
(kali@kali)-[~/ICDFA-Forensics-Lab]
$ binwalk evidence/Ch01InChap01.dd | tee results/binwalk_dd.txt
```

DECIMAL	HEXADECIMAL	DESCRIPTION
---------	-------------	-------------

Figure 10 — `binwalk evidence/J_ub_law.jpg` output: JPEG image data (EXIF standard) at offset 0, TIFF image data at offset 12, and a Copyright string "Copyright 1999 Adobe Systems Incorporated" at decimal offset 26844 (0x68DC) (Part D).

```

(kali@kali)~[ICDFA-Forensics-Lab]
$ file results/manual_extracted_image1.jpeg
md5sum results/manual_extracted_image1.jpeg
results/manual_extracted_image1.jpeg: Microsoft Word 2007+
580f02bf94a5adfed4b35dd82a48ee3e results/manual_extracted_image1.jpeg

(kali@kali)~[ICDFA-Forensics-Lab]
$ binwalk -e evidence/File_carving.docx -C results/binwalk_extracted
ls -R results/binwalk_extracted

DECIMAL      HEXADECEMIAL  DESCRIPTION
0            0x0           Zip archive data, at least v2.0 to extract, compressed size: 358, uncompressed size: 1364, name: [Content_Types].xml
927          0x39F         Zip archive data, at least v2.0 to extract, compressed size: 239, uncompressed size: 590, name: _rels/.rels
1727         0x6BF         Zip archive data, at least v2.0 to extract, compressed size: 328, uncompressed size: 1149, name: word/_rels/document.xml.rels
2377         0x949         Zip archive data, at least v2.0 to extract, compressed size: 1458, uncompressed size: 4120, name: word/document.xml
3882         0xF2A         Zip archive data, at least v1.0 to extract, compressed size: 85046, uncompressed size: 85046, name: word/media/image1.jpeg
88980        0x15B94       Zip archive data, at least v2.0 to extract, compressed size: 1571, uncompressed size: 6795, name: word/theme/theme1.xml
90602        0x161EA       Zip archive data, at least v2.0 to extract, compressed size: 1092, uncompressed size: 2785, name: word/settings.xml
91741        0x1665D       Zip archive data, at least v2.0 to extract, compressed size: 265, uncompressed size: 497, name: word/webSettings.xml
92056        0x16798       Zip archive data, at least v2.0 to extract, compressed size: 369, uncompressed size: 749, name: docProps/core.xml
92736        0x16A40       Zip archive data, at least v2.0 to extract, compressed size: 2927, uncompressed size: 28961, name: word/styles.xml
95708        0x175DC       Zip archive data, at least v2.0 to extract, compressed size: 453, uncompressed size: 1261, name: word/fontTable.xml
96209        0x177D1       Zip archive data, at least v2.0 to extract, compressed size: 388, uncompressed size: 731, name: docProps/app.xml

WARNING: One or more files failed to extract: either no utility was found or it's unimplemented

results/binwalk_extracted:
_File_carving.docx-0.extracted _File_carving.docx.extracted

results/binwalk_extracted/_File_carving.docx-0.extracted:
0.zip '[Content_Types].xml' docProps _rels word

results/binwalk_extracted/_File_carving.docx-0.extracted/docProps:
app.xml core.xml

results/binwalk_extracted/_File_carving.docx-0.extracted/_rels:

results/binwalk_extracted/_File_carving.docx-0.extracted/word:
document.xml fontTable.xml media _rels settings.xml styles.xml theme webSettings.xml

results/binwalk_extracted/_File_carving.docx-0.extracted/word/media:
image1.jpeg

```

Figure 11 — Manual dd extraction from File_carving.docx using offset 3882: the file command reports "Microsoft Word 2007+" instead of a JPEG — the offset pointed at the ZIP container header, not the JPEG byte stream. Subsequently, binwalk -e evidence/File_carving.docx -C results/binwalk_extracted performs automatic extraction (Part D — challenge documented).

```

results/binwalk_extracted:
_File_carving.docx-0.extracted _File_carving.docx.extracted

results/binwalk_extracted/_File_carving.docx-0.extracted:
0.zip '[Content_Types].xml' docProps _rels word

results/binwalk_extracted/_File_carving.docx-0.extracted/docProps:
app.xml core.xml

results/binwalk_extracted/_File_carving.docx-0.extracted/_rels:

results/binwalk_extracted/_File_carving.docx-0.extracted/word:
document.xml fontTable.xml media _rels settings.xml styles.xml theme webSettings.xml

results/binwalk_extracted/_File_carving.docx-0.extracted/word/media:
image1.jpeg

results/binwalk_extracted/_File_carving.docx-0.extracted/word/_rels:
document.xml.rels

results/binwalk_extracted/_File_carving.docx-0.extracted/word/theme:
theme1.xml

results/binwalk_extracted/_File_carving.docx.extracted:
0.zip '[Content_Types].xml' docProps _rels word

results/binwalk_extracted/_File_carving.docx.extracted/docProps:
app.xml core.xml

results/binwalk_extracted/_File_carving.docx.extracted/_rels:

results/binwalk_extracted/_File_carving.docx.extracted/word:
document.xml fontTable.xml media _rels settings.xml styles.xml theme webSettings.xml

results/binwalk_extracted/_File_carving.docx.extracted/word/media:
image1.jpeg

results/binwalk_extracted/_File_carving.docx.extracted/word/_rels:
document.xml.rels

results/binwalk_extracted/_File_carving.docx.extracted/word/theme:
theme1.xml

```

Figure 11 (cont.) — Binwalk auto-extraction layout: results/binwalk_extracted/_File_carving.docx.extracted/ reveals the OOXML container structure including [Content_Types].xml, docProps/, _rels/, and word/ subdirectories. The embedded image lives at word/media/image1.jpeg (Part D).

```
(kali@kali)-[~/ICDFA-Forensics-Lab]
$ mmls evidence/120M/usb_fat_carving.001 | tee results/usb_mmls.txt
DOS Partition Table
Offset Sector: 0
Units are in 512-byte sectors

    Slot      Start      End      Length    Description
000:  Meta      0000000000  0000000000  0000000001  Primary Table (#0)
001:  _____ 0000000000  0000000127  0000000128  Unallocated
002:  000:000  0000000128  0000243711  0000243584  Win95 FAT16 (0x0e)

(kali@kali)-[~/ICDFA-Forensics-Lab]
$
(kali@kali)-[~/ICDFA-Forensics-Lab]
$ fsstat -o 65536 evidence/120M/usb_fat_carving.001 | tee results/usb_fsstat.txt
Possible encryption detected (High entropy (7.58))

(kali@kali)-[~/ICDFA-Forensics-Lab]
$ fsstat -o 65536 evidence/120M/usb_fat_carving.001
Possible encryption detected (High entropy (7.58))

(kali@kali)-[~/ICDFA-Forensics-Lab]
$ cat results/hash_pre.txt
a117773bcf1fc88ec0ab8e0a349fbbcb evidence/Ch01InChap01.dd
83a360ac7f7e0ca318e5bfe39f95f137 evidence/J_ub_law.jpg
dfe7b5424e54cd1bf50d5df47aceeb3c evidence/120M.7z
3ce8053e4f3d9c8ab98b3aadb2480685efb8e4980d34297b83bd5a09b1a7b122 evidence/Ch01InChap01.dd
238ff34393c50e52c0e8b14fcff8ec7dc29e23914dbc435f8ef998d172a91468 evidence/J_ub_law.jpg
2d2a3d93c9ec65bcad9f89c9894429ea72caa8add07726a3b96ec9a6ab6a58ce evidence/120M.7z
9b1439fffd4a30d86a08299e659210590 evidence/File_carving.docx
a8c1d11f78a2df8a7f2e618320c7a3fc97438bc36c07de9fc12a2673588e38bf evidence/File_carving.docx
ba4a1d0ba49f4a6667b00a3b3e85e604 evidence/120M/usb_fat_carving.001
9bfe4b5634ade30764f0e581a4686930f7fab0472378595b789b0cdb248c91d9 evidence/120M/usb_fat_carving.001

(kali@kali)-[~/ICDFA-Forensics-Lab]
$
```

Figures 12 & 13 — mmls evidence/120M/usb_fat_carving.001 shows a Win95 FAT16 (0x0e) partition starting at sector 128 (byte offset 65536). fsstat -o 65536 then reports "Possible encryption detected (High entropy 7.58)" — the filesystem is encrypted or corrupted and cannot be browsed via metadata (Part E).

```
(kali@kali)-[~/ICDFA-Forensics-Lab]
$ grep -n "jpg" scalpel.conf
87: jpg y 5242880 \xff\xd8\xff???Exif \xff\xd9 REVERSE
88: jpg y 5242880 \xff\xd8\xff???JFIF \xff\xd9 REVERSE

(kali@kali)-[~/ICDFA-Forensics-Lab]
$ rm -rf results/scalpel_output
mkdir -p results/scalpel_output

(kali@kali)-[~/ICDFA-Forensics-Lab]
$ scalpel -c scalpel.conf -o results/scalpel_output evidence/120M/usb_fat_carving.001
Scalpel version 1.60
Written by Golden G. Richard III, based on Foremost 0.69.

Opening target "/home/kali/ICDFA-Forensics-Lab/evidence/120M/usb_fat_carving.001"

Image file pass 1/2.
evidence/120M/usb_fat_carving.001: 100.0% [*****] 119.0 MB 00:00 ETA
Allocating work queues ...
Work queues allocation complete. Building carve lists ...
Carve lists built. Workload:
jpg with header "\xff\xd8\xff\x3f\x3f\x3f\x45\x78\x69\x66" and footer "\xff\xd9" → 8 files
jpg with header "\xff\xd8\xff\x3f\x3f\x3f\x4a\x46\x49\x46" and footer "\xff\xd9" → 9 files
Carving files from image.
Image file pass 2/2.
evidence/120M/usb_fat_carving.001: 100.0% [*****] 119.0 MB 00:00 ETA
Processing of image file complete. Cleaning up ...
Done.
Scalpel is done, files carved = 17, elapsed = 1 seconds.

(kali@kali)-[~/ICDFA-Forensics-Lab]
$ ls -lh results/scalpel_output/
cat results/scalpel_output/audit.txt
ls -lh results/scalpel_output/jpg-*
total 12K
-rw-rw-r-- 1 kali kali 1.6K Sep  4 18:58 audit.txt
drwxrwxr-x 2 kali kali 4.0K Sep  4 18:58 jpg-0-0
drwxrwxr-x 2 kali kali 4.0K Sep  4 18:58 jpg-1-0

Scalpel version 1.60 audit file
Started at Fri Sep  4 18:58:22 2026
Command line:
```

Figure 14 — Scalpel configuration and execution: grep -n "jpg" scalpel.conf confirms the JPEG rules at lines 87–88 are uncommented, then scalpel -c scalpel.conf -o results/scalpel_output evidence/120M/usb_fat_carving.001 begins carving (Part F).

```

Scalpel version 1.60 audit file
Started at Fri Sep  4 18:58:22 2026
Command line:
scalpel -c scalpel.conf -o results/scalpel_output evidence/120M/usb_fat_carving.001

Output directory: /home/kali/ICDFA-Forensics-Lab/results/scalpel_output
Configuration file: scalpel.conf

Opening target "/home/kali/ICDFA-Forensics-Lab/evidence/120M/usb_fat_carving.001"

The following files were carved:
File           Start           Chop           Length           Extracted From
00000010.jpg   3796992           NO            3148500           usb_fat_carving.001
00000009.jpg   425822            NO            4875802           usb_fat_carving.001
00000008.jpg   335872            NO            4965752           usb_fat_carving.001
00000002.jpg   6938624           NO            6868             usb_fat_carving.001
00000001.jpg   5285888           NO            1659604           usb_fat_carving.001
00000000.jpg   3897344           NO            3048148           usb_fat_carving.001
00000004.jpg   15427584          NO            4223822           usb_fat_carving.001
00000003.jpg   12881920          NO            4256310           usb_fat_carving.001
00000006.jpg   19638272          NO            5229050           usb_fat_carving.001
00000005.jpg   17121280          NO            4248530           usb_fat_carving.001
00000007.jpg   21358592          NO            5183267           usb_fat_carving.001
00000016.jpg   36671488          NO            2889262           usb_fat_carving.001
00000015.jpg   36571136          NO            2989614           usb_fat_carving.001
00000014.jpg   36393610          NO            3167140           usb_fat_carving.001
00000013.jpg   36352448          NO            3208302           usb_fat_carving.001
00000012.jpg   35590996          NO            3969754           usb_fat_carving.001
00000011.jpg   35350242          NO            4210508           usb_fat_carving.001

Completed at Fri Sep  4 18:58:23 2026
results/scalpel_output/jpg-0-0:
total 27M
-rw-rw-r-- 1 kali kali 3.0M Sep  4 18:58 00000000.jpg
-rw-rw-r-- 1 kali kali 1.6M Sep  4 18:58 00000001.jpg
-rw-rw-r-- 1 kali kali 6.8K Sep  4 18:58 00000002.jpg
-rw-rw-r-- 1 kali kali 4.1M Sep  4 18:58 00000003.jpg
-rw-rw-r-- 1 kali kali 4.1M Sep  4 18:58 00000004.jpg
-rw-rw-r-- 1 kali kali 4.1M Sep  4 18:58 00000005.jpg
-rw-rw-r-- 1 kali kali 5.0M Sep  4 18:58 00000006.jpg
-rw-rw-r-- 1 kali kali 5.0M Sep  4 18:58 00000007.jpg

```

Figure 15 — Scalpel audit.txt file showing the full audit trail: 17 JPEG files carved (00000000.jpg through 00000016.jpg) with byte start offsets, chop lengths, and source image (usb_fat_carving.001). Total runtime: ~1 second (Part F).

```

results/scalpel_output/jpg-1-0:
total 32M
-rw-rw-r-- 1 kali kali 4.8M Sep  4 18:58 00000008.jpg
-rw-rw-r-- 1 kali kali 4.7M Sep  4 18:58 00000009.jpg
-rw-rw-r-- 1 kali kali 3.1M Sep  4 18:58 00000010.jpg
-rw-rw-r-- 1 kali kali 4.1M Sep  4 18:58 00000011.jpg
-rw-rw-r-- 1 kali kali 3.8M Sep  4 18:58 00000012.jpg
-rw-rw-r-- 1 kali kali 3.1M Sep  4 18:58 00000013.jpg
-rw-rw-r-- 1 kali kali 3.1M Sep  4 18:58 00000014.jpg
-rw-rw-r-- 1 kali kali 2.9M Sep  4 18:58 00000015.jpg
-rw-rw-r-- 1 kali kali 2.8M Sep  4 18:58 00000016.jpg

(kali@kali)~/ICDFA-Forensics-Lab
$ file results/scalpel_output/jpg-0-0/*.jpg
results/scalpel_output/jpg-0-0/00000000.jpg: JPEG image data, Exif standard: [TIFF image data, big-endian, direntries=7, orientation=upper-left, xresolution=98, yresolution=106, resolutionunit=2, software=Adobe Photoshop 21.1 (Macintosh), datetime=2020:04:03 14:35:44], baseline, precision 8, 1920x1080, components 3
results/scalpel_output/jpg-0-0/00000001.jpg: JPEG image data, Exif standard: [TIFF image data, big-endian, direntries=7, orientation=upper-left, xresolution=98, yresolution=106, resolutionunit=2, software=Adobe Photoshop 21.1 (Macintosh), datetime=2020:04:03 11:54:50], baseline, precision 8, 1920x1080, components 3
results/scalpel_output/jpg-0-0/00000002.jpg: JPEG image data, Exif standard: [TIFF image data, big-endian, direntries=12, width=16550, height=9313, bps=0, PhotometricInterpretation=RGB, orientation=upper-left]
results/scalpel_output/jpg-0-0/00000003.jpg: JPEG image data, Exif standard: [TIFF image data, big-endian, direntries=7, orientation=upper-left, xresolution=98, yresolution=106, resolutionunit=2, software=Adobe Photoshop 21.1 (Macintosh), datetime=2020:04:03 14:39:54], baseline, precision 8, 1920x1080, components 3
results/scalpel_output/jpg-0-0/00000004.jpg: JPEG image data, Exif standard: [TIFF image data, little-endian, direntries=16, width=4928, height=3280, bps=2, PhotometricInterpretation=RGB, manufacturer=NIKON CORPORATION, model=NIKON D4, orientation=upper-left], baseline, precision 8, 1920x1080, components 3
results/scalpel_output/jpg-0-0/00000005.jpg: JPEG image data, Exif standard: [TIFF image data, big-endian, direntries=7, orientation=upper-left, xresolution=98, yresolution=106, resolutionunit=2, software=Adobe Photoshop 21.1 (Macintosh), datetime=2020:04:21 10:36:43], baseline, precision 8, 1920x1080, components 3
results/scalpel_output/jpg-0-0/00000006.jpg: JPEG image data, Exif standard: [TIFF image data, big-endian, direntries=7, orientation=upper-left, xresolution=98, yresolution=106, resolutionunit=2, software=Adobe Photoshop 21.1 (Macintosh), datetime=2020:04:23 15:24:33], baseline, precision 8, 1920x1080, components 3
results/scalpel_output/jpg-0-0/00000007.jpg: JPEG image data, Exif standard: [TIFF image data, big-endian, direntries=12, width=1920, height=1080, bps=0, PhotometricInterpretation=RGB, orientation=upper-left], baseline, precision 8, 1920x1080, components 3

(kali@kali)~/ICDFA-Forensics-Lab
$

```

Figure 16 — file command verification of recovered JPEGs in jpg-1-0/ directory: each carved file is identified as "JPEG image data, Exif standard" with TIFF metadata, Adobe Photoshop 21.1 (Macintosh) software tag, datetime 2020:04:03, and 1920x1080 dimensions. This confirms the carved artifacts are valid photographic images, not false-positive signature matches (Part F).

2.7 Analysis and Findings

The findings below are organized by methodology part. Each subsection interprets the raw output produced in Section 2.5 and explains its significance in the context of the lab objectives.

Part A — Hex Analysis

The JPEG header was successfully identified as **ffd8** (SOI) and the footer as **ffd9** (EOI). The reconstructed image's MD5 and SHA-256 hashes matched the original perfectly, proving that the xxd round-trip (dump → reconstruct) is lossless. This validates xxd as a reliable tool for both inspection and bit-level reconstruction of binary evidence.

Part B — Metadata Strings

Metadata strings revealed the following provenance information embedded in the JPEG EXIF segment:

Field	Value
Camera Make	NIKON CORPORATION
Camera Model	NIKON D4
Software	Adobe Photoshop 21.1 (Macintosh)
Author / Artist	HOWARD KORN
Subject	2013, campus, Merrick School of Business
GPS Data	Not found in this image

Part C — TSK Filesystem Analysis

Ch01InChap01.dd was confirmed as a raw FAT12 superfloppy (no partition table) with a sector size of 512 bytes. **fls** identified 8 deleted files and 4 system files. **icat** successfully extracted a Microsoft Access Database from inode 5, and **blkcat** extracted the DOS boot sector from block 0. These results confirm that TSK remains effective when the filesystem is intact and readable.

Part D — Binwalk Embedded Content

Binwalk identified File_carving.docx as a ZIP archive containing an embedded JPEG at offset 3882 (ZIP header) and 3934 (actual JPEG data). Manual extraction using **dd** with offset 3934 successfully carved a valid JPEG (360×540). Automatic extraction via **binwalk -e** also recovered the file. This demonstrates that OOXML containers (DOCX, XLSX, PPTX) are ZIP archives whose embedded media can be carved without Office applications.

Part E — USB Image (High Entropy)

The USB image (usb_fat_carving.001) contained a Win95 FAT16 partition starting at sector 128. However, **fsstat** reported "High entropy (7.58)", indicating that the filesystem was encrypted or corrupted. Standard filesystem tools (**fls**, **icat**) were therefore unusable on this image — the metadata layer had been effectively destroyed.

CRITICAL FINDING

A Shannon entropy of 7.58 bits/byte is statistically indistinguishable from random data or strong encryption. No filesystem structure can be recovered from such a region — raw carving is the only viable path.

Part F — Scalpel Carving Result

Despite the high-entropy / encrypted filesystem, Scalpel successfully carved 17 files (8 Exif, 9 JFIF) directly from the raw image bytes. The **file** command confirmed these were valid JPEGs. File 00000004.jpg contained metadata linking it to the NIKON D4 camera, proving these were original photographic images — not random byte sequences that happened to match JPEG signatures. This result validates the file-signature-based approach of Scalpel: it operates on the raw bytes, completely bypassing the (corrupted) filesystem layer.

Carved File	Type	Notes
00000000.jpg — 00000007.jpg	JPEG / Exif	8 files — contain EXIF metadata
00000008.jpg — 00000016.jpg	JPEG / JFIF	9 files — JFIF header variant
Total carved	17 JPEGs	All verified as valid images via file command

2.8 Challenges and Solutions

Three significant challenges were encountered during the investigation. Each is documented below with the diagnostic command that identified the root cause and the corrective action taken. These challenges are instructive because they represent common failure modes in real-world forensic work.

Challenge 1 — Scalpel initially carved 0 files

Observation: Scalpel returned zero carved files on the first run, despite the image clearly containing JPEG data.

Diagnostic command:

```
grep -n "jpg" scalpel.conf
```

Resolution: Inspection revealed the JPEG rules in scalpel.conf were commented out. Uncommented the rules and re-ran Scalpel — 17 files carved successfully.

Challenge 2 — Manual dd extraction produced a "Microsoft Word 2007+" file instead of a JPEG

Observation: Using offset 3882 (the ZIP header reported by Binwalk) as the dd skip value produced an invalid JPEG.

Diagnostic command:

```
grep -abo '$\xff\xd8\xff' evidence/File_carving.docx
```


Resolution: The actual JPEG byte signature (ffd8ff) was located at offset 3934, not 3882. Using the correct offset produced a valid 360×540 JPEG.

Challenge 3 — fsstat / fls reported "High entropy (7.58)" on the USB image

Observation: The USB FAT16 filesystem was unreadable, blocking all metadata-based recovery.

Diagnostic command:

```
fsstat -o 65536 evidence/120M/usb_fat_carving.001
```

Resolution: Bypassed the filesystem entirely using Scalpel raw carving, which scans file signatures directly in the byte stream rather than relying on filesystem metadata.

2.9 Conclusion

This lab successfully demonstrated that when filesystem metadata is missing, damaged, or encrypted, standard forensic tools such as **fls** or **icat** are ineffective. However, raw data carving techniques using tools like **Scalpel** can bypass these limitations by scanning for file signatures directly within the binary data — the filesystem layer is irrelevant to a carver.

The lab successfully reconstructed a JPEG from a hex dump (Part A), extracted embedded content from a DOCX file (Part D), and carved 17 deleted JPEG files from a corrupted FAT16 USB image (Part F). Each of these outcomes reinforces the central principle of file carving: **file signatures are ground truth; filesystem metadata is only a convenience**. This principle becomes operationally critical in adversarial scenarios where suspects deliberately damage or encrypt filesystems to impede investigation.

Beyond the specific tools used, the lab reinforced a methodological discipline: compute hashes at every stage, verify reconstructed artifacts by hash, and escalate from filesystem-aware tools to raw carving only when metadata proves unreliable. This escalation ladder is the standard operating procedure for professional digital forensics work.

2.10 Recommendations

Based on the findings and challenges encountered during this investigation, the following actionable recommendations are proposed for both future lab work and real-world forensic engagements:

- **Hash discipline:** Always calculate and document MD5 and SHA-256 hashes of evidence before *and* after analysis. Hashes are the only mathematical proof of evidentiary integrity.
- **Escalation ladder:** When filesystem metadata is suspected to be corrupted or encrypted, proceed directly to raw carving rather than spending time attempting filesystem repair.
- **Exact byte offsets:** Use **grep -abo** to locate exact file signatures when manually carving from containers such as DOCX, XLSX, ZIP, or other OOXML formats. Do not trust container-reported offsets blindly.

- **Configuration verification:** Verify all configuration files (such as **scalpel.conf**) using **grep** before execution. A single commented-out rule can silently produce zero results.
- **Tool triangulation:** Cross-validate carving results with multiple tools (Binwalk, Scalpel, foremost) and confirm output with the **file** command. False positives are common in raw carving.

2.11 References

- [1] The Sleuth Kit (TSK) Documentation. Retrieved from <https://www.sleuthkit.org/>
- [2] Scalpel Documentation. Kali Linux Tools. Retrieved from <https://www.kali.org/tools/scalpel/>
- [3] Binwalk Documentation. ReFirmLabs. Retrieved from <https://github.com/ReFirmLabs/binwalk>
- [4] National Institute of Standards and Technology (NIST). Guide to Digital Forensics.
- [5] International Cybersecurity and Digital Forensics Academy. Lab Assignment Template Guide (v1.0).